PROJECT REPORT

Mini Game with Python

Stone • Paper • Scissor

Submitted To:

Gowthami Mam

Submitted By:

Vijayalaxmi

Besant Technologies

Python Programming | Beginner Project | 2024

Acknowledgement

I sincerely thank **Gowthami Mam** and **Besant Technologies** for their invaluable guidance, constant support, and encouragement throughout this project. Their expertise and mentorship have been instrumental in shaping my understanding of Python programming.

This project provided me with a great opportunity to apply theoretical knowledge in a practical environment. Working on the Stone Paper Scissor game helped me improve my logical thinking, problem-solving abilities, and coding skills. I am grateful for every learning experience this project offered.

> *"This work helped improve my practical understanding of Python programming and gave me confidence to build real-world applications."*

Index

What is Python?

Python is a **high-level, easy-to-learn programming language** known for its simple syntax and readability. It is widely used for web development, software development, data analysis, artificial intelligence, automation, and game development. Python is beginner-friendly and supports object-oriented and functional programming concepts.

When and Where Python Was Introduced

Python was created by **Guido van Rossum** and was first introduced in **1991**. It was developed at **Centrum Wiskunde & Informatica (CWI)** in the Netherlands.

Key Concepts of Python

1. Variables

Variables are used to store data values in a program.

```
name = "Vijayalaxmi"
```

2. Data Types

Python supports different data types like integer, string, float, and boolean.

```
age = 25
price = 99.5
```

3. Conditional Statements

Conditional statements like if, elif, and else are used for decision making.

```
if age > 18:
print("Adult")
```

4. Loops

Loops are used to repeat a block of code multiple times.

```
for i in range(5):
print(i)
```

5. Functions

Functions help reuse code and make programs organized.

```
def greet():
print("Hello")
```

6. Modules

Modules are files containing reusable Python code.

```
import random
```

Uses of Python

Area	Description
Data Analysis	Used for analysing and visualizing data using Pandas and NumPy.
Web Development	Used to build websites and web applications using Django and Flask.
Artificial Intelligence	Used in machine learning and AI projects.
Automation	Automates repetitive tasks and saves time.
Game Development	Used to create simple games and applications.
Software Development	Used for desktop applications and system software.
Cyber Security	Used for ethical hacking and security testing tools.

Python is popular because it is **simple, powerful, and versatile** for many real-world applications.

Project Introduction

Stone Paper Scissor is a classic hand game enjoyed worldwide for generations. In this project, the game has been recreated as an interactive Python program where a user competes against the computer. The computer randomly selects one of the three moves — stone, paper, or scissor — and the program then determines the winner based on the game rules.

Project Objective

The main goals of this project are:

- Understand how to use the **random module** in Python.
- Practice **conditional statements** (if, elif, else) for decision-making.
- Learn **user input handling** and input validation.
- Work with **Python lists** to store and access data.
- Build practical **logic and problem-solving** skills.
- Create a fun, real-time interactive application as a beginner.

Game Rules

Choice	Beats	Because
Stone	Scissor	Stone blunts the scissor
Paper	Stone	Paper covers the stone
Scissor	Paper	Scissor cuts the paper
Tie	—	Both chose the same move

Key Concepts Used in Project

The following Python concepts were applied while building this game:

1. Random Module

The **random** module is a built-in Python library. **random.choice()** is used to allow the computer to randomly pick stone, paper, or scissor — making each round unique and fair.

2. Conditional Statements

if, **elif**, and **else** compare the user's choice against the computer's choice and determine the outcome: win, lose, or tie.

3. User Input & Validation

The **input()** function captures the player's choice. The **.lower()** method makes it case-insensitive. Validation checks the choice is within the valid list before the game proceeds.

4. Python Lists

A **list** stores the three valid choices: *['stone', 'paper', 'scissor']*. Used for input validation (with the **in** operator) and for **random.choice()**.

5. print() Function

The **print()** function displays the computer's choice and the game result to the user.

6. f-Strings

f-strings (formatted string literals) are used to display the computer's choice dynamically: *print(f"Computer chose: {computer_choice}")*

Key Concepts (Continued) — Program Flow

The program follows a clear, step-by-step flow from start to finish:

Step	Action
Step 1	Import the **random** module
Step 2	Create a **list** of valid choices: stone, paper, scissor
Step 3	Accept the **user's input** and convert to lowercase
Step 4	**Validate** the input — if invalid, print error and stop
Step 5	Use **random.choice()** to generate the computer's move
Step 6	**Compare** both choices using conditional statements
Step 7	**Display** the result: You Win / Computer Wins / It's a Tie

Additional Python Concepts

- **input() Function:** Captures keyboard input from the user at runtime.
- **str.lower():** Converts any string to lowercase — ensures 'Stone' and 'stone' are treated equally.
- **not in operator:** Checks whether a value does **not** exist in a list — used for input validation.
- **Logical Operators (and / or):** Used to combine multiple conditions in the elif block to check all winning scenarios.

Source Code

Below is the complete Python source code for the Stone Paper Scissor game:

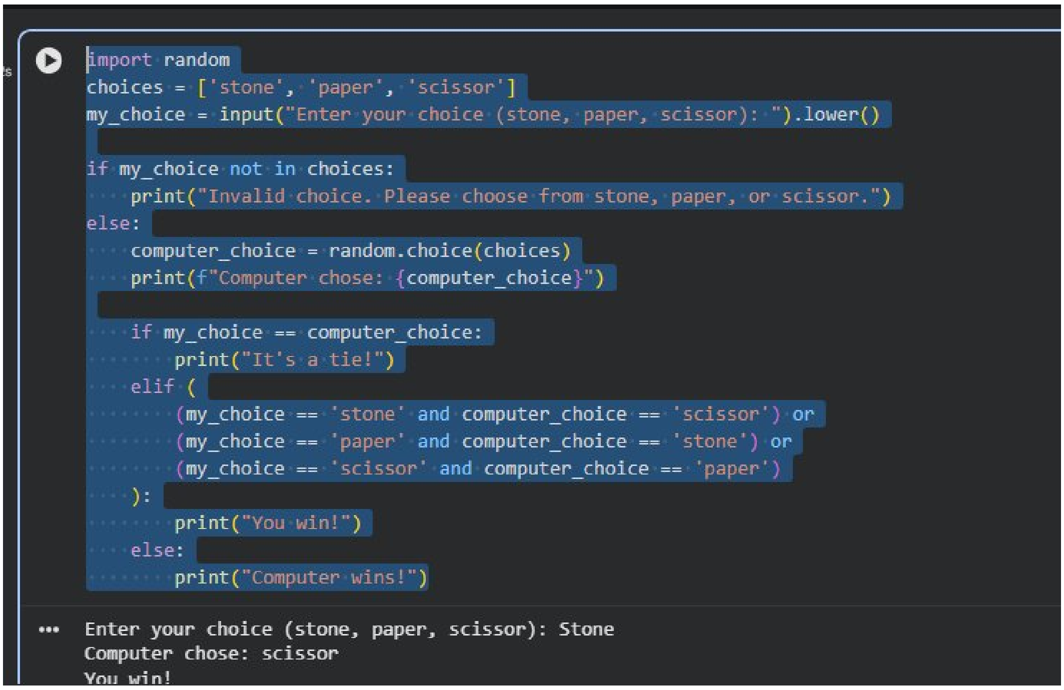
```
import random

choices = ['stone', 'paper', 'scissor']
my_choice = input("Enter your choice (stone, paper, scissor): ").lower()

if my_choice not in choices:
print("Invalid choice. Please choose from stone, paper, or scissor.")
else:
computer_choice = random.choice(choices)
print(f"Computer chose: {computer_choice}")

if my_choice == computer_choice:
print("It's a tie!")
elif (
(my_choice == 'stone' and computer_choice == 'scissor') or
(my_choice == 'paper' and computer_choice == 'stone') or
(my_choice == 'scissor' and computer_choice == 'paper')
):
print("You win!")
else:
print("Computer wins!")
```

Line-by-Line Explanation:

- **Line 1:** Imports the random module to enable random selection.
- **Line 3:** Creates a list of all valid game choices.
- **Line 4:** Takes user input and normalises it to lowercase.
- **Lines 6–7:** Validates input; exits gracefully if invalid.
- **Lines 9–10:** Computer randomly picks its move and displays it.
- **Lines 12–21:** Compares choices and prints Win / Tie / Lose result.

Program Output — Screenshot

The screenshot below shows the program running in a Python IDE. The user entered **'Stone'**, the computer chose **'scissor'**, and the program correctly declared **'You win!'**




```
Sample Output:
Enter your choice (stone, paper, scissor): Stone
Computer chose: scissor
You win!
```

Conclusion

The **Stone Paper Scissor Game using Python** is a simple and interactive mini project developed to understand basic programming concepts in Python. This project helps beginners learn important concepts such as the random module, conditional statements, user input, lists, and decision making in a practical way.

The game allows the user to play against the computer, making the program fun and engaging. By developing this project, programming logic and problem-solving skills are greatly improved. Overall, this project is a good example of how Python can be used to create simple real-time applications in an easy and effective manner.

Key Learnings

✔ Used the **random module** to make the game unpredictable.

✔ Applied **if / elif / else** logic to determine winners.

✔ Practiced **input validation** for robust user interaction.

✔ Managed valid choices using **Python lists**.

✔ Understood **program flow** from input to output.

✔ Gained confidence in creating real-world Python programs.

This project is a beginner-friendly mini game that improves programming logic, conditional thinking, and practical Python knowledge. It lays a strong foundation for more advanced Python projects in the future.